Program = passive instructions
Process = active execution of those instructions.

1) Process Concept :-

A program is a passive entity consisting of a set of instructions stored on secondary storage, usually as an executable file.

• Process :-

A process is a program in execution.
A process is not only the program code. It also contains:

- Current value of the program counter (IPC).
- Contents of CPU registers.
- Current data and variables.
- Stack
- Heap
- Other resources associated with its execution.

• Example :-

Chrome.exe stored on the disk is a program When Chrome is loaded into memory and starts executing, it becomes a process.

* One program can have multiple process.
* Processes are the unit of work that the OS schedules and manages.

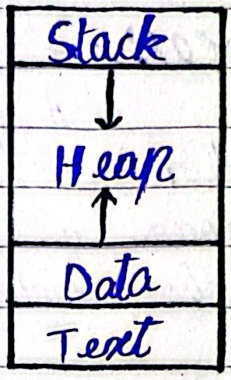
Scanned with CamScanner

2) Process in Memory :-

When a program is loaded into memory and becomes a process, its memory is logically divided into several sections.

The main section are:

1. Text Section
2. Data Section
3. Heap
4. Stack



Main Memory

1. Text Section :-

The text section contains the executable program code, i.e., the machine instructions that the process executes.

It is sometimes called the code section.

2. Data Section :-

The data section contains:

- Global variables
- Static variables

These variables exist for the lifetime of the process.

3. **Heap :-**

The heap is the area of memory used for dynamic memory allocation during program execution.

Memory can be allocated and released from the heap as required by the program.

4. **Stack :-**

The stack stores temporary informat required during function execution, such as

- Function parameters
- Return addresses
- Local variables

The stack changes as functions are called and return.

* The process also has a current execution state represented by information such as the program counter and CPU registers. Therefore two processes executing the same program can still have different data, stack, heap, and execution states.

⊙ A process is therefore more than just its program code.

Scanned with CamScanner

3) Process Life Cycle :-

A process does not remain in one state throughout its execution. As it is created, scheduled, executed, waits for resources, and finishes, it moves through different process states.

The exact names may vary between OS, but the following five basic states are commonly used.

Process State = The current activity / status of a process at a particular moment.

• Seven Basic Process States :-

~~A process does not remain in one state throughout its execution As it create~~

In the seven-state process model, a process can exist in the following seven states:

1. New
2. Ready
3. Running
4. Waiting
5. Suspended Ready
6. Suspended Waiting
7. Terminated

Scanned with CamScanner

1. **New** :-

The process is being created.

- The process has just been started.
- The OS creates the process and prepares the required information and resources.
- The process has not yet started. executing instructions on the CPU.

Example: You double-click a program → OS begins creating its process → New state.

2. **Ready** :-

The process is ready to execute but is waiting for the CPU to be assigned to it

- The process has everything necessary to execute.
- It is waiting for the OS scheduler to give it CPU time.
- Multiple processes can be in the ready state simultaneously.
- The process is normally present in main memory.

Scanned with CamScanner

Ready → waiting for CPU
Waiting → waiting for something else

3. Running :-

The process is currently executing instructions on the CPU.

- The CPU is executing instructions belonging to this process.
- A process normally enters Running from Ready after being selected by the scheduler.
- On a single CPU, only one process can be in the Running state at a particular moment.

4. Waiting :-

The process is waiting for some event or resource before it can continue execution. Common reasons include:

- Waiting for I/O to complete
- Waiting for user input
- Waiting for a file or resource
- Waiting for another event.

While the process is waiting, the CPU can be assigned to another process.

Scanned with CamScanner

Suspended Ready = ready to run, but temporarily out of main memory.

5. Suspended Ready :-

A process is in the Suspended Ready state when it is ready to execute but has been temporarily suspended and removed from main memory.

- The process is otherwise ready to execute.
- It cannot currently be scheduled because it is not in main memory.
- The OS may suspend it to free-memory space or manage system resources.
- When it is resumed and brought back into main memory, it returns to the Ready state.

* It is not waiting for I/O or another event. Its main issue is that it has been suspended

6. Suspended Waiting :-

A process is in the Suspended Waiting state when it is waiting for an event or resource and has also been temporarily removed from main memory.

- The process was already in the Waiting state.
- The OS suspends it and removes it from main memory.
- It remains unable to execute because the required event or resource is still unavailable.

Scanned with CamScanner

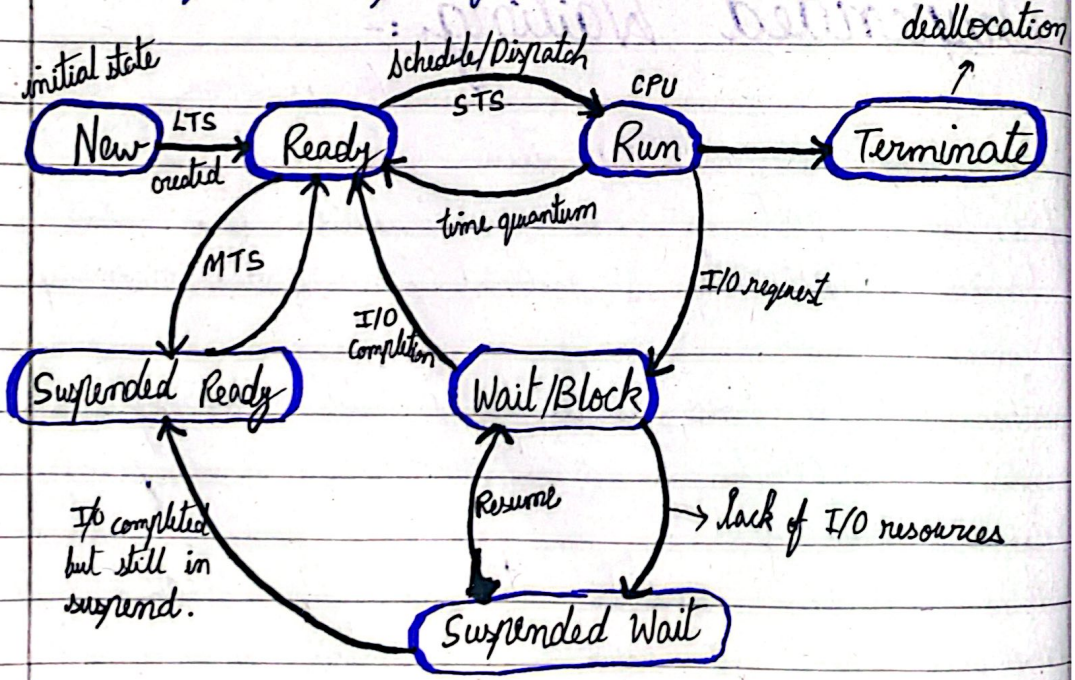
- When the process is resumed, it can return to the waiting state.
- When the required event eventually occurs, it can move to the Ready state.

7. Terminated :-

The process has finished execution or has been terminated by the operating system.

- The process has completed its execution.
- The OS releases the resources allocated to the process.
- The process eventually ceases to exist.

* A process may also be terminated by the OS before completing its normal execution.



* PCB allows OS to manage multiple processes independly and resume ~~that~~ them correctly after they stop running.

4) Process Control Block (PCB):-

A PCB is a data structure maintained by the OS for every process. It contains the important information the OS needs to identify, manage, schedule, suspend, and resume a process.

- Every process has its own PCB
- The PCB is also called Task Control Block (TCB).

• Why Does the OS Need a PCB?

The OS manages many processes at the same time. For each process, the OS must know things such as:

- What state is the process in?
- What values were in its CPU registers?
- Where should its execution continue?
- What is its scheduling priority?
- What memory belongs to it?
- What files does it have open?
- How much CPU time has it used?

The OS stores this information in the process 's PCB.

* The PCB becomes especially important when the CPU switches from one process to another.

Scanned with CamScanner

• Information Stored in a PCB :-

A PCB contains several categories of information associated with a particular process.

1. Process State
2. Process Identification / Process Number
3. Program Counter
4. CPU registers
5. CPU - Scheduling information
6. Memory - Management Information
7. Accounting Information
8. I/O status Information

1. Process State :-

Indicates the current state of the process, such as New, Ready, Running, Waiting or Terminated.

2. Process ID (PID) :-

A unique identification number assigned to the process by the OS.

3. Program Counter :-

Stores the address of the next instruction to be executed by the process.

Scanned with CamScanner

4. CPU Registers :-

Stores the current values for the CPU register of the process.

5. CPU Scheduling Information :-

Contains information used by the scheduler, such as priority and scheduling queue information.

6. Memory Management Information :-

Contains information about the memory allocated to the process, such as page tables or memory limits.

7. Accounting Information :-

Stores information about CPU time used, process execution time, and other resource usage.

8. I/O Status Information :-

Contains information about the I/O devices and files being used by the process.

Scanned with CamScanner

5) Process Scheduling :-

Process Scheduling is the activity of selecting processes for execution and managing their movement among the different scheduling queues.

The main objective is to select a process that is ready to execute and allocate the CPU to it.

For a single-processor system, only 1 process can be running at a time. Other processes must wait in appropriate queues.

- Scheduling Queues :-

During its lifetime, a process moves through different queues according to its current state. The main queues are:

1. Job Queue
2. Ready Queue
3. Waiting / Device Queue

a) Job Queue :-

The job queue contains all processes that have entered the system and are waiting to be admitted for execution.

Scanned with CamScanner

- It contains processes waiting to be brought into main memory.
- It is associated with secondary storage.
- The long-term scheduler selects processes from the job queue and admits them into main memory.

b) Ready Queue :-

The ready queue contains processes that are:

- In main memory
- Ready to execute
- Waiting for the CPU.

The short-term scheduler selects a process from the ready queue and gives it the CPU.

c) Waiting Queue :-

When running process request I/O or must wait for a particular event, it cannot continue using the CPU. It moves to a waiting queue, also called a device queue when waiting for a specific I/O devices.

Each I/O device can have its own device queue

6) Schedulers :-

A scheduler is an OS component that selects processes for execution and decides when they should be admitted to or removed from memory.

The three main types of schedulers are:

1. Long - Term Scheduler
2. Short - Term Scheduler
3. Medium - Term Scheduler

Each scheduler performs a different role in process management.

1. Long-Term Scheduler :-

- Selects processes from the Job Queue.
- Loads selected processes into main memory and places them in the Ready Queue.
- Controls which programs are admitted for execution.
- Works less frequently.

Flow : Job Queue → Long term Scheduler → Ready Queue.

Scanned with CamScanner

2. Short-Term Scheduler :-

- Selects a processes from the Ready Queue.
- Allocates the CPU to the selected process.
- Runs very frequently, so it must be fast.
- Uses scheduling criteria / algorithms to decide which process runs next.

Flow : Ready Queue → Short-Term Scheduler → CPU.

3. Medium-Term Scheduler :-

- Temporarily removes processes from main memory when memory resources are insufficient.
- Stores them on secondary storage.
- Later brings them back into memory when required.
- This process is called swapping.

Flow : Main Memory ↔ Medium-Term Scheduler ↔ Secondary Storage.

Scanned with CamScanner

7) Process Scheduling Times :-

- **Arrival Time (AT):** The time at which a process enters the Ready Queue.
- **Burst Time (BT):** The total CPU time required to execute the process.
  - Does not include wasting time.
- **Completion Time (CT):** The time at which the process completes its execution.
- **Turnaround time (TAT):** Total time spent by a process from arrival to completion.
  Formula: TAT = CT - AT
- **Waiting Time (WT):** Total time for which a process waits for the CPU.
  Formula: WT = TAT - BT
- **Response Time (RT):** Time from the process's arrival until it first gets the CPU.
  Formula: First CPU start time - AT

Scanned with CamScanner

8) CPU Scheduling :-

It is the process of selecting a process from the Ready Queue and allocating the CPU to it.

- In multiprogramming, when one process waits for I/O, the CPU can execute another process.
- The OS schedules processes to achieve maximum CPU utilization.
- This is performed by the Short-Term Scheduler.
- Scheduling algorithms determines which process gets the CPU next.

a) Uniprogramming :-

- Only one program executes at a time.
- If the process waits for I/O, the CPU remains idle.

b) Multiprogramming :-

- Multiple processes can be kept in memory.
- When one process waits for I/O, the CPU executes another process.
- Improves CPU utilization.

Scanned with CamScanner

9) CPU Scheduling Algorithms :-

CPU scheduling algorithms are the methods used by the OS to decide which process from the Ready Queue should be given the CPU next.

Different algorithms use different criteria, such as arrival order, burst time, priority, or time quantum.

1. First Come First Serve (FCFS) :-

- Processes are executed in the order they arrive in the Ready Queue.
- Non-preemptive.

2. Shortest Job First (SJF) :-

- Selects the process with the shortest Burst Time.
- Non-preemptive.

3. Shortest Remaining Time First (SRTF)

- Preemptive version of SJF.
- Selects the process with the shortest remaining Burst Time.

Scanned with CamScanner

- A running process can be preempted when a shorter process arrives.

4. Priority Scheduling :-

- CPU is allocated to the process with the highest priority.
- Can be preemptive or non-preemptive
- A lower numerical value represents high priority.

5. Round Robin (RR) :-

- Each process receives a fixed time quantum.
- If the process does not finish within its quantum, it is preempted and placed at the back of the Ready Queue.
- Suitable for fair sharing of CPU time.

6. Multilevel Queue :-

- Divides the Ready Queue into separate queues with different priorities.
- Processes are placed into queues according to their priority / type.
- Each queue can use its own scheduling algorithm.

Scanned with CamScanner

10) **Context Switching :-**

It is the procedure of saving the current state (context) of a running process and loading the saved state of another process so that execution can continue from the same point later.

It allows a single CPU to be shared among multiple processes.

- **Process Context :-**

The context of a process represents the information required to resume its execution.

The saved context is associated with the process through its PCB.

Important execution information include :

- Process state
- Program Counter
- CPU registers
- Other information required to resume execution.

Scanned with CamScanner

• Context Switching Procedure :-

Suppose P_1 is currently running and the CPU must execute P_2.

Steps :-

1. P_1 is currently executing
2. The OS saves the context/state of P_1
3. The saved context of P_1 is maintained in its PCB
4. The OS loads the saved context of P_2.
5. CPU starts/resumes execution of P_2.

When P_1 gets the CPU again, its previously saved context is loaded and execution resumes from where it stopped.

• Context Switching Triggers :-

There are three major triggers for context switching.

i) Multitasking :-

In a multitasking environment the CPU is switched from one process to another.

On a preemptive system, the scheduler may switch a processor out of the CPU.

Scanned with CamScanner

ii) **Interrupt Handling :-** When an interrupt occurs, the hardware changes part of the current context so that the interrupt can be handled.

Only the required portion of the context is changed to reduce the time needed to handle the interrupt.

iii) **User Mode & Kernal Mode Switching :-** A context switch may occur when the system needs to transition between:

User Mode ⟷ Kernel Mode

This occurs when execution requires operating-system-level services or processing.

* Context switching is necessary for multi-tasking.

11) Process Management :-

It is the OS function responsible for managing processes throughout their life cycle, from creation ~~and~~ to termination.

In multiprogramming or multitasking system, multiple processes may be active at the same time. The OS must manage their CPU execution, resources, communication, synchronization and termination

- Purpose of Process Management :-

1. Manage process creation and termination.
2. Allocate CPU to processes through CPU scheduling.
3. Manage process communication.
4. Synchronize cooperating processes.
5. Handle deadlocks.
6. Manage resources required by process.
7. Maintain efficient execution of multiple processes.

Process Management in Multiprogramming :-

In a single-task system, process management is simpler because

only one process is active at a time. In multiprogramming, multiple processes are active, making management more complex.

The OS must manage their execution, shared resources, communication, and synchronization.

- Main Advantages :-

- Better CPU utilization
- Better system responsiveness.
- CPU can execute another process while one waits for I/O.

- Process Management Tasks :-

1. Process Creation & Termination : Creates processes, assigns a PID, sets up the PCB, and releases resources when a process terminates.

2. CPU Scheduling : Selects processes for CPU execution.

3. Deadlock Handling : Handles situations where processes cannot proceed because of cyclic dependencies on resources.

4. Inter-Process Communication : Allows cooperating processes to communicate through shared memory or message passing.

Scanned with CamScanner

5. Process Synchronization: Coordinates processes to ensure controlled access to shared resources.

• Advantages of Process Management :-

- Running Multiple Program: Allows multiple applications to execute concurrently.
- Process Isolation: Helps prevent processes from interfering with one another.
- Fair Resource Use: Distributes CPU time and other resources among processes.
- Smooth Switching: Saves and loads process states to switch between processes.

• Overheads / Disadvantages :-

- Overhead: Requires additional CPU time and memory to manage processes and scheduling queues
- Complexity: Managing multiple processes and resources makes the OS more complex.
- Deadlocks: Processes may become unable to proceed while waiting for resources held by one another.
- Context - Switching Overhead: Saving and loading process states consumes time and CPU resources.

Scanned with CamScanner

12) Types of Process :-

Processes can be classified into two types:

a) Independent Process :-

It is a process whose execution is not affected by the execution of other processes.

An independent process operates on its own and does not share data or resources with other processes. Since it does not need to communicate or coordinate with other processes, IPC is not required.

b) Cooperating Process :-

It is a process that can interact with other processes and share data or resources. It's execution can be affected by the execution of other processes.

Cooperating processes need to communicate and coordinate their activities. IPC provides the mechanism for communication and synchronization between cooperating processes.

Scanned with CamScanner

13) Inter-Process Communication (IPC):-

IPC is a mechanism that allows process to communicate and exchange data or information with each other.

IPC is mainly required for cooperating processes because they may need to share information, coordinate their activities, and synchronize their execution.

• Why is IPC Required?

Process may need to communicate when they work together to perform a task. IPC helps processes to :

- Exchange information: Processes can send and receive required data.
- Share data and resources: Process can communicate information related to shared data or resources.
- Synchronize activities: Processes can coordinate the order and timing of their operations.
- Avoid conflicts: It help processes access shared resources in a controlled manner.
- Support cooperation: Processes can work together as part of a larger task.

Scanned with CamScanner

• IPC Models :-

The two fundamental models of Inter-Process Communication are:

1. Shared Memory
2. Message Passing

a) Shared Memory :-

It is an IPC method in which two or more processes communicate by accessing a shared area of memory.

Processes can write information to the shared memory and other processes can read the information.

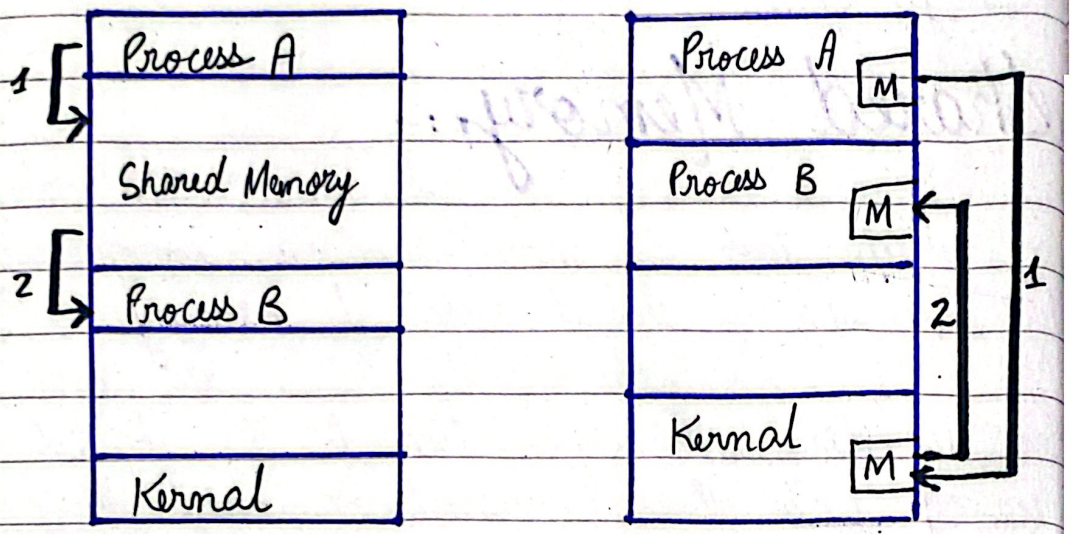
The implementation of shared memory communication depends on how the programmer designs the shared variables and memory usage.

b) Message Passing :-

It is an IPC method in which processes communicate and exchange information by sending and receiving messages through the OS kernal.

Scanned with CamScanner

Unlike shared memory, processors do not directly access a common memory area. Instead, a process uses system calls provided by the OS to request the kernal to send or receive a message.



14) Operations ON Process :-

A process operation is an operation performed by the OS when a process is in a particular state or moves from one state to another.

The main operations are:

1. Process Creation
2. Process Dispatch
3. Process Pre-emption
4. Process Blocking
5. Process Termination

i) Process Creation :-

It is the operation of creating a new process in the system.

A newly created process is initially associated with the : New State. After the necessary process information and resources are prepared, it can be placed in the Ready State.

• Cause of Process Creation :-

A process may be created for several reasons :

• System Initialization :-

When the computer starts, the OS creates various system & background processes required for the system to operate.

• User Request :-

When a user starts a program or application, the OS creates a process to execute the program.

• Child Process Creation :-

A running process can create another process using a process - creation system call. The existing process becomes the parent process, while the newly created process becomes its child process.

Scanned with CamScanner

• Batch System :-

The OS may initiate a new process when a batch job is submitted for execution.

• Parent and Child Process :-

A process can create another process.

- The process that create another process is called the Parent Process.
- The newly created one is called Child Process.
- A child process has one parent.
- A parent process may have multiple children.

• fork() System Call :-

In systems such as UNIX/LINUX, a process can create a child process using the fork() system call.

The fork() operation creates a new child process based on the calling process.

- Parent and child processes have the same memory image, open files, and environment strings.
- They have distinct address spaces.

ii) Process Dispatch :-

It occurs when a process in the Ready state is selected for execution and is given control of the CPU.

The selection of the process is made by the scheduler according to the scheduling algorithm being used.

The component responsible for giving the selected process the CPU is called the dispatcher.

When a process is dispatched, its required execution information, such as the CPU context stored in PCB, is loaded so that the process can continue or begin execution.

• Process Dispatch May Occur When :-

- CPU becomes idle and processes are present in the Ready Queue.
- Time quantum of the currently running process expires.
- A high priority process enters the system.
- A hardware interrupt occurs.

Scanned with CamScanner

* Preemption takes the CPU away from the currently running process.

iii) Process Pre-emption :-

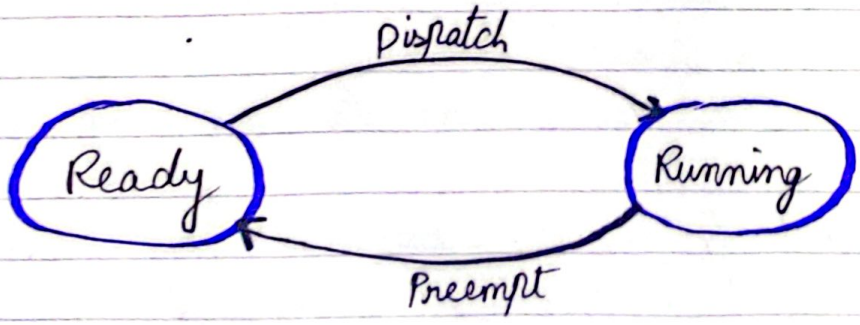
Process pre-emption is the operation in which the OS interrupts a currently running process and takes the CPU away from it, allowing another process to be selected for execution.

The pre-empted process is generally moved from the Running State back to the Ready State.

- Causes of Pre-emption :-

1. Time quantum of the running process expires
2. A higher-priority process enters the Ready Queue.
3. A hardware interrupt occurs.

* The context of the outgoing process is stored in its PCB so that the process can later resume from the same point.



Scanned with CamScanner

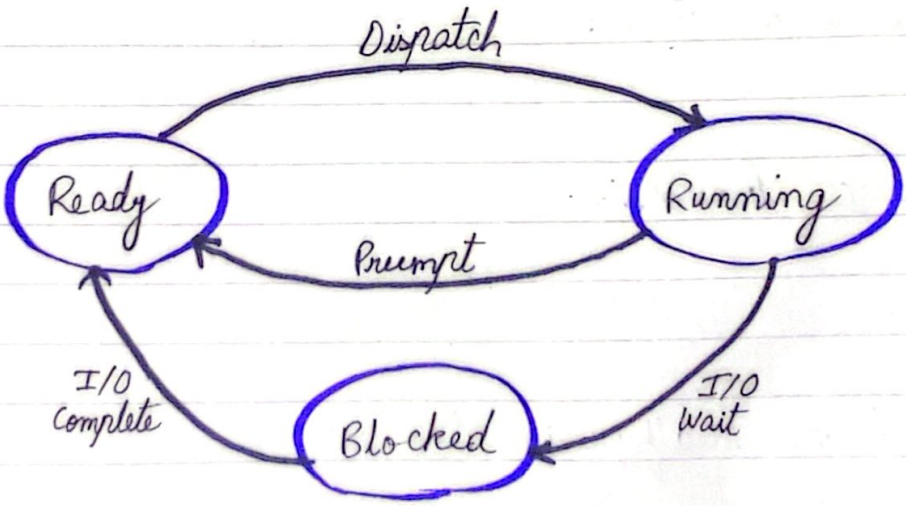
iv) Process Blocking :-

It occurs when a currently running process cannot continue its execution because it must wait for a particular event or resource.

The process is moved from the Running state to the Waiting / Blocked state.

The most common reason for blocking is waiting for an I/O operation to complete.

When the required event occurs, the process moves to the ready state, then it can later be dispatched to the CPU.



Scanned with CamScanner

v) Process ~~Blocking~~, Termination :-

It is the operation of ending the execution of a process.

After termination, the process ceases to exist, and the OS releases the resources associated with it.

The resources may include memory, open files, and other resources allocated to the process.

• Causes of Process Termination :-

1. **Normal Completion**: The process completes execution of its last instruction.
2. **Parent Termination**: A parent process may terminate a child process if the child's task is no longer required.
3. **Service Errors**: The OS may terminate a process if it encounters certain service or system related errors that prevent the process from continuing correctly.
4. **Hardware Malfunction**: A hardware failure or malfunction may cause a process to terminate